

P2355R1: Postfix fold expressions

Audience: EWG

S. Davis Herring <herring@lanl.gov>

August 10, 2021

History

Since [r0](#):

- Prohibited inconsistent folds like $(x[\dots[abc]])$
- Fixed syntax in `index` example
- Used *initializer-clause* instead of redundant *assign-or-braced-init-list*

Introduction

Fold expressions work with binary operators, but not with unary operators: you can write `!!...!x`, but there's still only one `x`. However, there are other kinds of operators to which they might apply. In particular, there are several plausible use cases for the postfix operators `[]` and `()` (function call). This paper proposes extending the *fold-expression* syntax to support these two operators. The rationale for the syntactical structure is also presented.

Syntax

The appropriate syntax may not be immediately obvious, but it can be constructed by analogy to the binary operator case. In particular, the subscripting operator is almost an ordinary binary operator already. (Infamously, the built-in operator is commutative: `1["$?"]` has the same meaning as `"$?"[1]`.) Consider how it would be supported as a binary operator `@` (to which we will not ascribe an associativity): if `x @ a` is equivalent to `x[a]`, then `x[a][b][c]` is equivalent to `((x @ a) @ b) @ c`, which is the result of the binary left fold `(x @ ... @ abc)`. Bearing in mind the implied grouping, that fold suggests the syntax

```
(x[...][abc])
```

for the case of recursive indexing.

Similarly, `(xyz[...][a])` or `(xyz @ ... @ a)` means `x @ (y @ (z @ a))` or `x[y[z[a]]]`: a lookup with a sequence of indirections. Note the corresponding nesting in the fold-expression form. Furthermore, `(xyz[...])` is the unary right fold `(xyz @ ...)`, which is `x[y[z]]`, and the unary left fold `(...[xyz])` means `x[y][z]`; these have somewhat narrower applicability, since elements of the same pack must be usable both as containers and as indices. As *postfix-expressions* have the highest precedence, parentheses are strictly required only for the unary cases, but it is prudent to require them in all cases for consistency.

The corresponding cases for the call operator are obvious: a left fold applies each result to the next argument as a function (in a fashion similar to method chaining), while a right fold composes functions in a pack. However, function calls can have multiple arguments. Unary folds cannot construct such calls, but binary folds can be generalized in a straightforward fashion:

```
(f(...)(abc,x))    // f(a,x)(b,x)(c,x)
(f(...)(abc,xyz))  // f(a,x)(b,y)(c,z)
```

```
(fgh(...(a,x)))    // f(g(h(a,x)))
```

If the subscripting operator [is extended to support multiple arguments](#), the obvious analogous syntax can apply there as well.

Since both these operators support a *braced-init-list* in the right-hand operand(s), the binary folds should do so as well. The unary left fold (... [{abc,x}]) seems plausible but would result in the meaningless {a,x} [{b,x}] [{c,x}]. Similarly, (... (abc,x)) would mean **a,x**(b,x)(c,x), with a bare *expression-list*.

Were it desired, even the cast operators would work in just the binary right fold case:

```
(static_cast<TUV>(static_cast<...>(a)))    // static_cast<T>(static_cast<U>(static_cast<V>(a)))
(TUV{... {a,x}})                          // T{U{V{a,x}}}}
((TUV)(... )a)                            // (T)(U)(V)a
```

Placement new would similarly support just the binary left fold:

```
(new (new (x) ...) TUV)                    // new (new (new (x) T) U) V
(new (new (x) ...) T(abc,y))                // new (new (new (x) T(a,y)) T(b,y)) T(c,y)
```

These are illustrated here for completeness and to demonstrate the generality of the approach; they are certainly not proposed.

Motivation

[]

The syntactic investigation for this proposal was instigated by the discussion of multi-parameter subscripting operators. The notion of folding over [] was [explicitly mentioned](#) in proposals on the subject, and it could serve as an alternative to support for multiple subscripting arguments:

P2128R3

```
// arr defines a multi-parameter
operator[]
decltype(auto) index(auto &arr, auto ...ii)
{return arr[ii...];}
```

this proposal

```
// arr defines a C++20 proxy-based
operator[]
decltype(auto) index(auto &arr, auto ...ii)
{return (arr[...][ii]);}
```

Even if support for operator[] having multiple parameters is added, this proposal serves to support existing types, including those like arrays and `std::vector` that are unlikely to support multiple indexing. It also supports the very different right fold case.

()

The convenience of fold expressions (especially when the successive subexpressions might have different types), combined with their restriction to operators, has led to [common usage of workarounds](#) involving expressing a function as an operator defined for a type that exists purely to allow a fold. This proposal allows function objects to be used instead, reducing syntactic overhead. Some such folds may be used to emulate expressions of the form `f(a, f(b, f(c, d)))`, which are still not directly available because they are folds over *f itself* rather than over ().

C++20

this proposal

```

namespace detail {
    template<class F>
    struct call {
        F &&f;
        template<class T>
        decltype(auto) operator|(T &&t) const
        {return std::forward<F>(f)
        (std::forward<T>(t));}
    };
}

```

```

template<class T, class X>
decltype(auto) nest_tuple(T &&t, X &&x) {
    return std::apply
        ([&x]<class ...TT>(TT &&...tt) ->
        decltype(auto)
        {return (detail::call<TT>
        {std::forward<TT>(tt)} | ...
        | std::forward<X>(x));},
        std::forward<T>(t));
}

```

```

namespace detail {
    template<class T>
    struct smaller {
        T &t;
        template<class U>
        auto& operator|(const smaller<U> &c)
        const {
            if constexpr(sizeof(T)<sizeof(U))
                return *this;
            else return c;
        }
    };
}

```

```

auto& smallest(const auto &...aa)
{return (detail::smaller{aa} | ...).t;}

```

or (at the cost of duplicating what could be a long signature)

```

auto& smallest(const auto &a) {return a;}
auto& smallest(const auto &a, const auto
&...aa) {
    auto &b=smallest(aa...);
    if constexpr(sizeof a < sizeof b) return
    a;
    else return b;
}

```

```

template<class T, class X>
decltype(auto) nest_tuple(T &&t, X &&x) {
    return std::apply
        ([&x]<class ...TT>(TT &&...tt) ->
        decltype(auto)
        {return (std::forward<TT>(tt)(...
        (std::forward<X>(x))));},
        std::forward<T>(t));
}

```

```

auto smallest(const auto &a, const auto
&...aa) {
    // f(a1, f(a2, ...a)) -- note reordering
    return ([&](auto &x) -> auto& {
        if constexpr(sizeof aa < sizeof x)
            return aa;
        else return x;
    })(...(a));
}

```

Proposal

For C++23, support unary and binary folds over the operators `[]` and `()`, with the syntax summarized below for the latter. For `[]`, restrict to single right-hand operands with no top-level comma operators (which might be a

braced-init-list) unless that restriction is removed generally (e.g., by P2128). Do not support empty unary folds for either operator for lack of an appropriate identity element. (One could argue for some sort of identity function that preserves value category for `()` (as a left identity like `void()` is for `,`), but that seems drastically inventive.) This change does not affect the meaning of well-formed C++20 programs; the syntax is ungrammatical there.

	Example	Meaning
Binary left fold	<code>(f(...)({abc, 0}, x))</code> <code>(f(...)(abc, xyz))</code>	$f(\{a, 0\}, x)(\{b, 0\}, x)(\{c, 0\}, x)$ $f(a, x)(b, y)(c, z)$
Binary right fold	<code>(fgh(...({a, 0}, x)))</code> <code>(fgh(...()))</code>	$f(g(h(\{a, 0\}, x)))$ $f(g(h()))$
Unary left fold	<code>(... (abc))</code>	$a(b)(c)$
Unary right fold	<code>(abc (...))</code>	$a(b(c))$

Wording

Relative to N4885.

[**expr.prim.fold**]

Change paragraph 1:

- A fold expression performs a fold of a pack ([temp.variadic]) over a binary **or postfix** operator.

fold-expression:

```
( cast-expression fold-operator ... )
( ... fold-operator cast-expression )
( cast-expression fold-operator ... fold-operator cast-expression )
( postfix-expression [ ... ] [ initializer-clause ] ).
( postfix-expression [ ... [ initializer-clause ] ] ).
( postfix-expression [ ... ] ).
( ... [ assignment-expression ] ).
( postfix-expression ( ... ) ( expression-listopt- ) ).
( postfix-expression ( ... ( expression-listopt- ) ).
( postfix-expression ( ... ) ).
( ... ( assignment-expression ) ).
```

[...]

[Drafting note: If the subscripting operator comes to accept 0 or more subscripts, we can use *expression-list_{opt}* instead of *initializer-clause*. — end drafting note]

Change paragraph 2:

- An expression of the form **A fold-expression that begins with** `( ... op e )` where *op* is a fold-operator is called a *unary left fold*. **An expression of the form** `(e op A fold-expression that ends with ... )` where *op* is a fold-operator, **[...]**, or **(...)** is called a *unary right fold*. Unary left folds and unary right folds are collectively called *unary folds*. In a unary fold, the *cast-expression*, **postfix-expression, or assignment-expression** shall contain an unexpanded pack ([temp.variadic]).

Change paragraph 3:

- An expression of the form $(e_1 \text{ op1 } \dots \text{ op2 } e_2)$ where *op1* and *op2* are *fold-operators* Any other *fold-expression* is called a *binary fold*. If a binary fold, *op1* and *op2* shall be the same contains two *fold-operator*, and either *e1* shall contain an unexpanded pack or *e2* shall, they shall be the same. A binary fold has two operands, each an expression, an *expression-list* or nothing, or a *braced-init-list*; exactly one of them shall contain an unexpanded pack, but not both. If *e2* the second contains an unexpanded pack, the expression is called a *binary left fold*; it shall not end with *)* or *]*. If *e1* the first contains an unexpanded pack, the expression is called a *binary right fold*; it shall not be formed with *(...)* or *[...]*. [Example:

[...]

— end example]

[temp.variadic]

Change bullet (5.12):

- In a *fold-expression* ([*expr.prim.fold*]); the pattern is the *east-expression* *operand* that contains an unexpanded pack.

Change paragraph 10:

- The instantiation of a *fold-expression* produces:

[...]

In each case, *op* is the *fold-operator*, *N* is the number of elements in the pack expansion parameters, and each E_i is generated by instantiating the pattern and replacing each pack expansion parameter with its i^{th} element. If there is no *fold-operator*, *op* is a notional operator that applies the subscription operator if the *fold-expression* has a *[* or the function call operator otherwise, such that $X \text{ op } Y$ is $X[Y]$ or $X(Y)$ respectively.

[Note: It is possible for *Y* to be a possibly empty expression list or a *braced-init-list*. — end note]

For a binary fold-expression, *E* is generated by instantiating the *east-expression* *operand* that did not contain an unexpanded pack.

[...]